

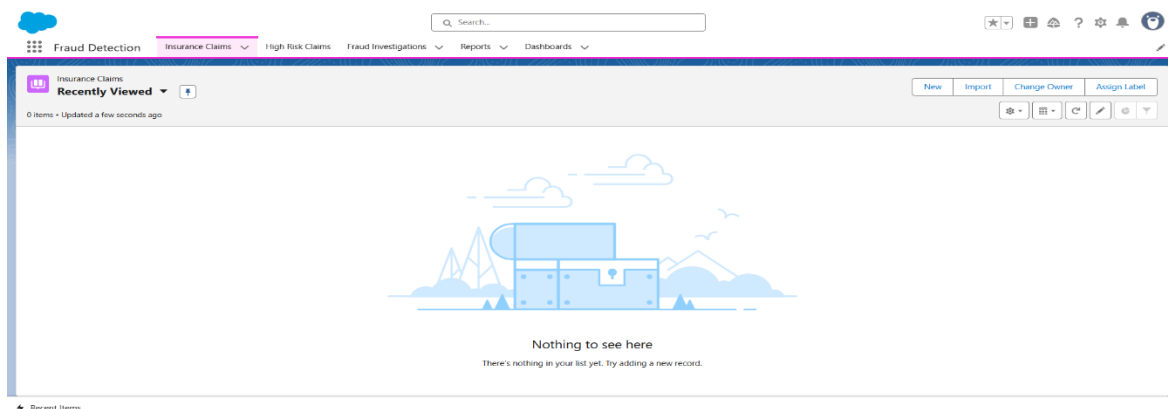
Phase 6: User Interface Development

Goal:

Build a user-friendly and interactive Salesforce Lightning interface for Fraud Detection, enabling Adjusters, Analysts, and Managers to access claims, view fraud scores, and take actions seamlessly.

1. Lightning App Builder – Fraud Detection App

- Created a new **Lightning App** named ***Fraud Detection***.
- Added navigation items: **Insurance Claims**, **Fraud Investigations**, **Reports**, and **Dashboards**.
- Created **Custom Tabs** for custom objects and added them to the app.
- **Used:**
 - To give project stakeholders a **dedicated workspace** to manage claims and fraud investigations without navigating multiple apps.
- **Steps Followed:**
 1. Setup → **App Manager** → **New Lightning App**.
 2. Enter *App Name*: Fraud Detection.
 3. Added navigation items (Insurance Claims, Fraud Investigations, Reports, Dashboards).
 4. Assigned the app to **Analyst, Manager, and Adjuster** profiles.

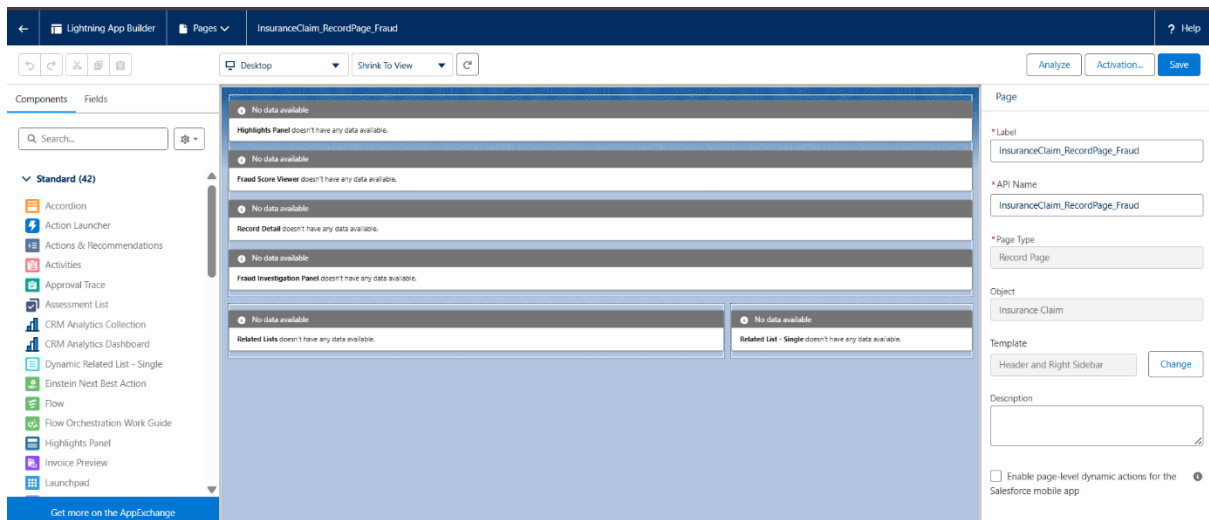


2. Record Pages – Claim Page Customization

- Customized the **Insurance Claim record page** with components:
 - *Highlights Panel* (Claim #, Status, Risk Score).
 - *Record Detail* (claim details).
 - *Related Lists* (documents, fraud investigations).
 - *Custom LWCs* → fraudScoreViewer, fraudInvestigationPanel.
- **Used:**
 - To provide a **single view of claim and fraud details** for faster decision-making.
- **Steps Followed:**
 1. Setup → **Lightning App Builder** → New Record Page.
 2. Selected **Insurance Claim** object.
 3. Dragged required components into layout.
 4. Added custom LWCs after deployment.
 5. Activated the page for the *Fraud Detection App*.

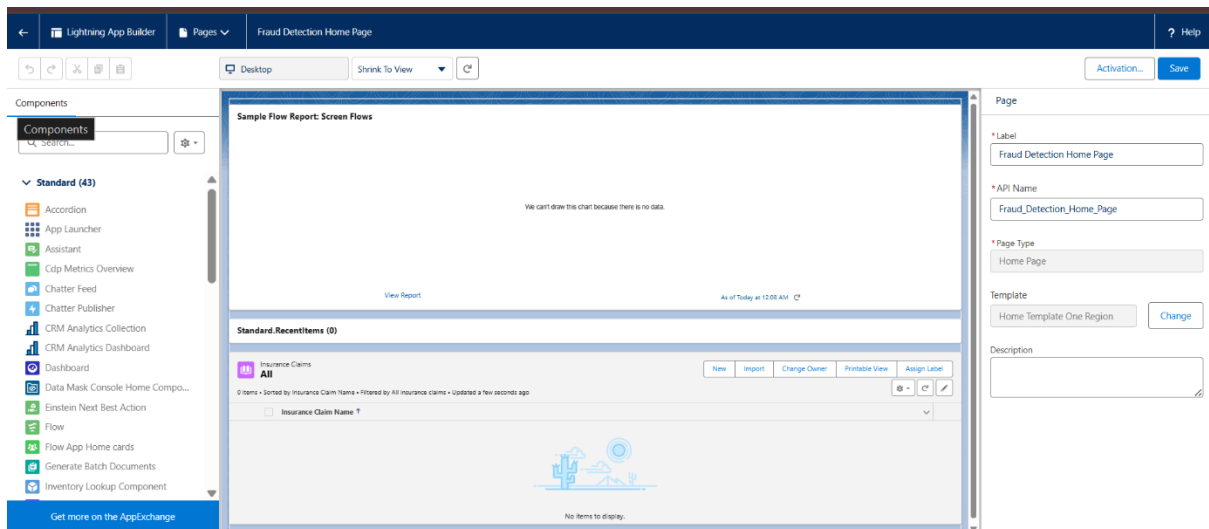
The screenshot displays the Lightning App Builder interface. At the top, the 'Lightning App Builder' header is visible. Below it, the 'Lightning Page' section shows the page name 'InsuranceClaim_RecordPage_Fraud' with 'Edit', 'Clone', and 'Delete' buttons. The 'Lightning Page Detail' section shows the page's information, including its name and label. Below this, the 'Assignments By App' section shows 'No Assignments to display'. The 'Assignments By App, Record Type, and Profile' section contains a table with the following data:

App	Record Type	Profile	Form Factor
Fraud Detection	Accident Claim	Claims Adjuster Profile	Desktop and phone
Fraud Detection	Medical Claim	Claims Adjuster Profile	Desktop and phone
Fraud Detection	Other Claim	Claims Adjuster Profile	Desktop and phone
Fraud Detection	Theft Claim	Claims Adjuster Profile	Desktop and phone
Fraud Detection	Accident Claim	Fraud Analyst Profile	Desktop and phone
Fraud Detection	Medical Claim	Fraud Analyst Profile	Desktop and phone
Fraud Detection	Other Claim	Fraud Analyst Profile	Desktop and phone
Fraud Detection	Theft Claim	Fraud Analyst Profile	Desktop and phone



3. Home Page Layouts:

- Created a Fraud Detection Home Page.
- Added components:
 - *Report Chart* → Fraud Trends.
 - *Recent Items*.
 - *List View* → High Risk Claims.
- **Used:**
 - To give Managers a dashboard-like overview of fraud risks and recent claims.
- **Steps Followed:**
 1. Setup → Lightning App Builder → New Home Page.
 2. Added components: *Report Chart*, *Recent Items*, *List View*.
 3. Activated for the *Fraud Detection App* and profiles.

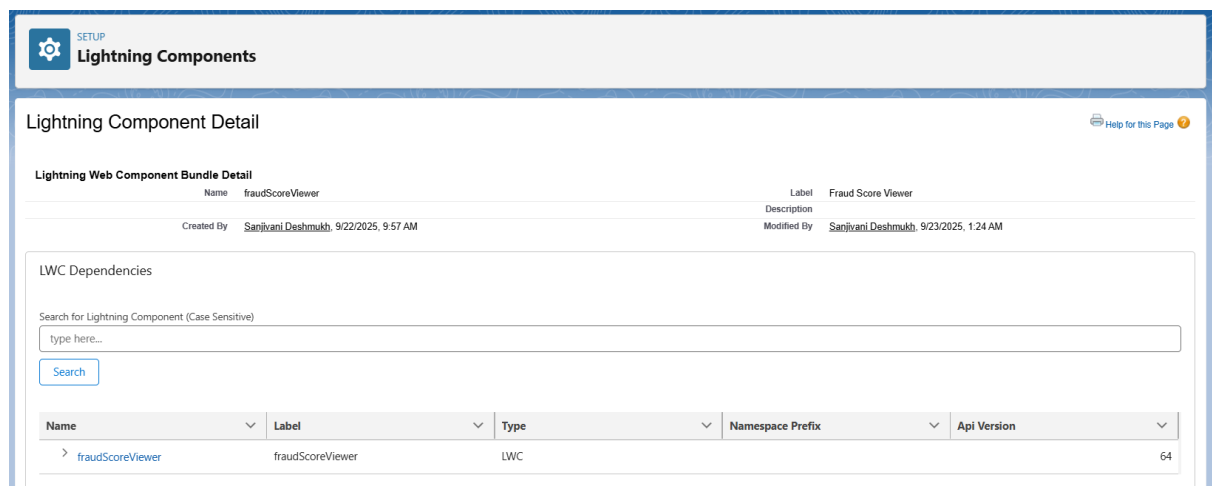


4. Lightning Web Components (LWCs)

Developed 3 LWCs to extend UI functionality:

(a) fraudScoreViewer

- **Purpose:** Displays real-time fraud score for a claim.
- **Usage:** Added to Insurance Claim Record Page.
- **Why:** Allows Analysts to instantly check fraud risk without running reports.



```
1 <template>
2   <lightning-card title="Claim Summary">
3     <div class="slds-m-around_medium">
4       <p><strong>Claim:</strong> {claimNumber}</p>
5       <p><strong>Status:</strong> {status}</p>
6       <p><strong>Amount:</strong> {amount}</p>
7       <p><strong>Risk Score:</strong> <lightning-badge label={riskScore}></lightning-badge>
8       <lightning-button label="Open Investigations" onclick={openInvestigations} class="slds-button--primary"></lightning-button>
9     </div>
10  </lightning-card>
11 </template>
12
```

```
1 import { LightningElement, api, wire } from 'lwc';
2 import { getRecord, getFieldValue } from 'lightning/uiRecordApi';
3 import CLAIM_NUMBER from '@salesforce/schema/Insurance_Claim__c.Claim_Number__c';
4 import CLAIM_AMOUNT from '@salesforce/schema/Insurance_Claim__c.Claim_Amount__c';
5 import RISK_SCORE from '@salesforce/schema/Insurance_Claim__c.Risk_Score__c';
6 import STATUS from '@salesforce/schema/Insurance_Claim__c.Status__c';
7
8 import { NavigationMixin } from 'lightning/navigation';
9
10 const FIELDS = [CLAIM_NUMBER, CLAIM_AMOUNT, RISK_SCORE, STATUS];
11
12 export default class FraudScoreViewer extends NavigationMixin(LightningElement) {
13   @api recordId;
14   @wire(getRecord, { recordId: '$recordId', fields: FIELDS })
15   claim;
16
17   get claimNumber() { return getFieldValue(this.claim.data, CLAIM_NUMBER) || '-'; }
18   get amount() { return getFieldValue(this.claim.data, CLAIM_AMOUNT) || '-'; }
19   get riskScore() { return getFieldValue(this.claim.data, RISK_SCORE) || '-'; }
20   get status() { return getFieldValue(this.claim.data, STATUS) || '-'; }
21
22   openInvestigations() {
23     this[NavigationMixin.Navigate]({
24       type: 'standard__recordRelationshipPage',
25       attributes: {
26         recordId: this.recordId,
27         relationshipApiName: 'Fraud_Investigations__r', // relationship name - verify in
28         actionName: 'view'
29       }
30     });
31   }
32 }
```

```

1 <?xml version="1.0" encoding="UTF-8"?>
2 <LightningComponentBundle xmlns="http://soap.sforce.com/2006/04/metadata">
3   <apiVersion>64.0</apiVersion>
4   <isExposed>true</isExposed>
5   <masterLabel>Fraud Score Viewer</masterLabel>
6   <targets>
7     <target>lightning__RecordPage</target>
8     <target>lightning__AppPage</target>
9   </targets>
10  <targetConfigs>
11    <targetConfig targets="lightning__RecordPage">
12      <property name="recordId" type="String" />
13    </targetConfig>
14  </targetConfigs>
15 </LightningComponentBundle>
16

```

(b) fraudInvestigationPanel

- **Purpose:** Shows related investigations for a claim, with option to create or update investigations.
- **Usage:** Added to record page layout.
- **Why:** Helps Analysts track fraud investigations directly from the claim page.

Lightning Component Detail

Lightning Web Component Bundle Detail

Name	Label
fraudInvestigationPanel	Fraud Investigation Panel

Created By: Sanjivani Deshmukh, 9/22/2025, 11:40 PM
Modified By: Sanjivani Deshmukh, 9/23/2025, 1:24 AM

LWC Dependencies

Search for Lightning Component (Case Sensitive)

type here...

Search

Name	Label	Type	Namespace Prefix	Api Version
> fraudInvestigationPanel	fraudInvestigationPanel	LWC		64

```
EXPLORER
  FRAU...
  .sf
  .sfdx
  .vscode
  config
  force-app\main\def...
  applications
  aura
  classes
  contentassets
  flexipages
  layouts
  lwc
    fraudInvestigation...
      > _tests_
    fraudInvestigatio...
    JS fraudInvestigatio...
    fraudInvestigatio...
  fraudScoreViewer
    > _tests_
    fraudScoreViewer...
    JS fraudScoreViewer...
    fraudScoreViewer...
  OUTLINE

<template>
1
2 <lightning-card title="Investigations">
3   <template if:true={invList}>
4     <template for:each={invList} for:item="inv">
5       <div key={inv.Id} class="slds-box slds-m-around_x-small">
6         <p><strong>ID:</strong> {inv.Name}</p>
7         <p><strong>Score:</strong> {inv.Fraud_Score__c}</p>
8         <p><strong>Status:</strong> {inv.Investigation_Status__c}</p>
9         <lightning-button variant="brand" label="Mark Closed" data-id={inv.Id} oncli
10       </div>
11     </template>
12   </template>
13   <template if:true={error}>
14     <div class="slds-text-color_error">{error}</div>
15   </template>
16 </lightning-card>
17 </template>
18
```

```
JS fraudInvestigationPanel.js
force-app > main > default > lwc > fraudInvestigationPanel > JS fraudInvestigationPanel.js > ...

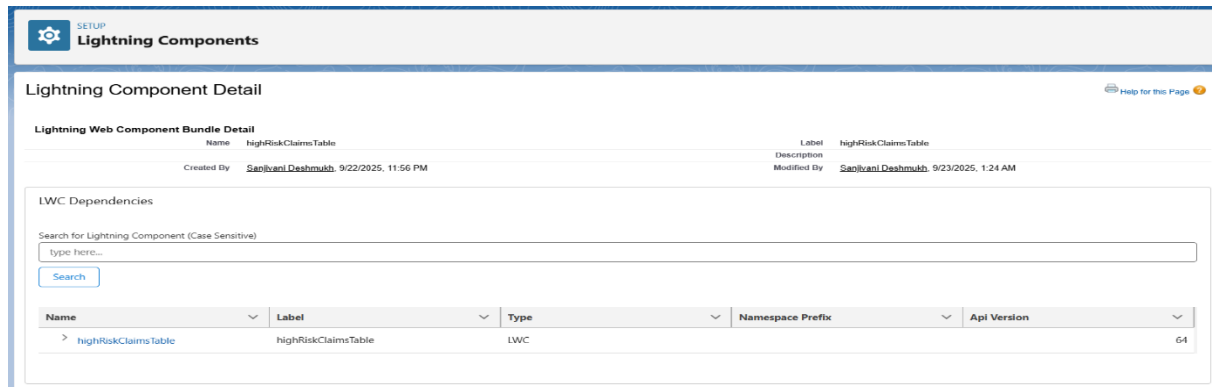
1 import { LightningElement, api, wire, track } from 'lwc';
2 import getInvestigationsByClaim from '@salesforce/apex/FraudLwcController.getInvestigationsByClaim';
3 import updateInvestigationStatus from '@salesforce/apex/FraudLwcController.updateInvestigationStatus';
4
5 export default class FraudInvestigationPanel extends LightningElement {
6   @api recordId;
7   @track invList;
8   @track error;
9
10   @wire(getInvestigationsByClaim, { claimId: '$recordId' })
11   wiredInvs({ error, data }) {
12     if (data) { this.invList = data; this.error = undefined; }
13     else if (error) { this.error = error.body ? error.body.message : error; this.invList = u
14   }
15
16   handleMarkClosed(event) {
17     const invId = event.target.dataset.id;
18     updateInvestigationStatus({ invId: invId, newStatus: 'Closed', notes: 'Closed via UI' })
19       .then(result => {
20         // dispatch event to parent to refresh
21         this.dispatchEvent(new CustomEvent('investigationupdated', { detail: { id: invId
22         return;
23       })
24       .catch(err => {
25         this.error = err.body ? err.body.message : err;
26       });
27   }
28 }
29
```

```
fraudInvestigationPanel.js-meta.xml
force-app > main > default > lwc > fraudInvestigationPanel > fraudInvestigationPanel.js-meta.xml > ...

1 <?xml version="1.0" encoding="UTF-8"?>
2 <LightningComponentBundle xmlns="http://soap.sforce.com/2006/04/metadata">
3   <apiVersion>64.0</apiVersion>
4   <isExposed>true</isExposed>
5   <masterLabel>Fraud Investigation Panel</masterLabel>
6   <targets>
7     <target>lightning__RecordPage</target>
8   </targets>
9 </LightningComponentBundle>
10
```

(c) highRiskClaimsTable

- **Purpose:** Displays all high-risk claims in a table view.
- **Usage:** Added to Home Page or Utility Bar.
- **Why:** Allows Managers to monitor all high-risk claims at a glance.

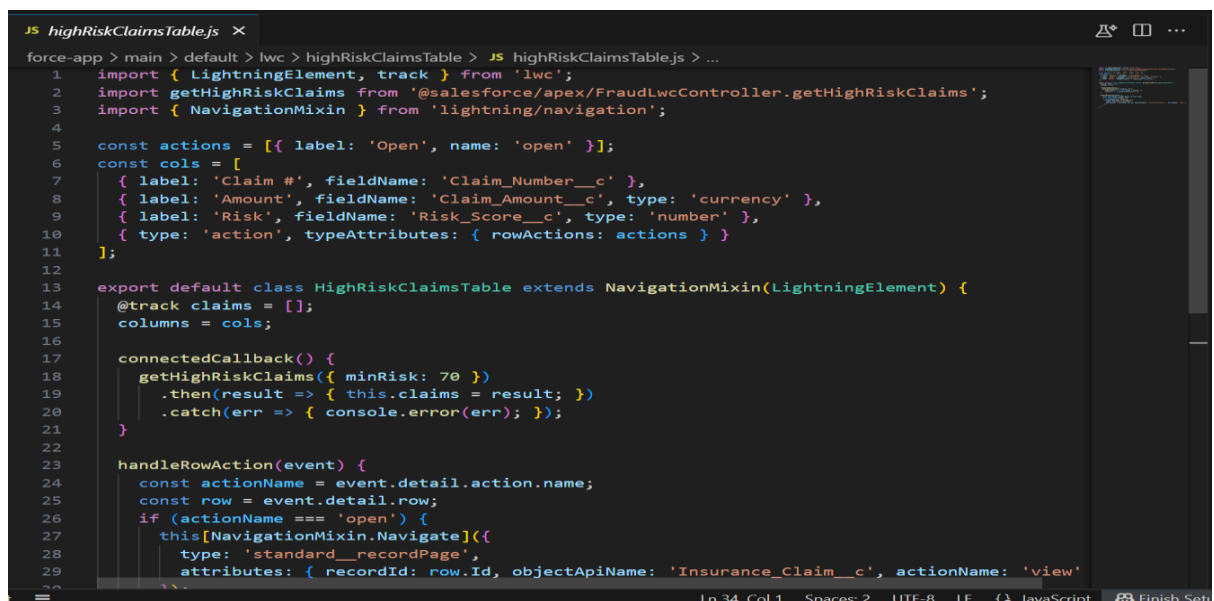


The screenshot shows the 'Lightning Component Detail' page in the Salesforce Lightning Component Builder. The component is named 'highRiskClaimsTable' and is labeled 'highRiskClaimsTable'. It was created by 'Santvani Dashedmuh' on 9/22/2025 at 11:56 PM and modified on 9/23/2025 at 1:24 AM. The page displays the 'LWC Dependencies' section, which includes a search bar and a table of dependencies. The table has columns for Name, Label, Type, Namespace Prefix, and Api Version. A single dependency is listed: 'highRiskClaimsTable' with a label of 'highRiskClaimsTable', type of 'LWC', and an API version of '64'.

Name	Label	Type	Namespace Prefix	Api Version
highRiskClaimsTable	highRiskClaimsTable	LWC		64



```
<? highRiskClaimsTable.html >
force-app > main > default > lwc > highRiskClaimsTable > <? highRiskClaimsTable.html > ...
1  <template>
2    <lightning-card title="High Risk Claims">
3      <lightning-datatable
4        data={claims}
5        columns={columns}
6        key-field="Id"
7        onrowaction={handleRowAction}>
8      </lightning-datatable>
9    </lightning-card>
10  </template>
11
```



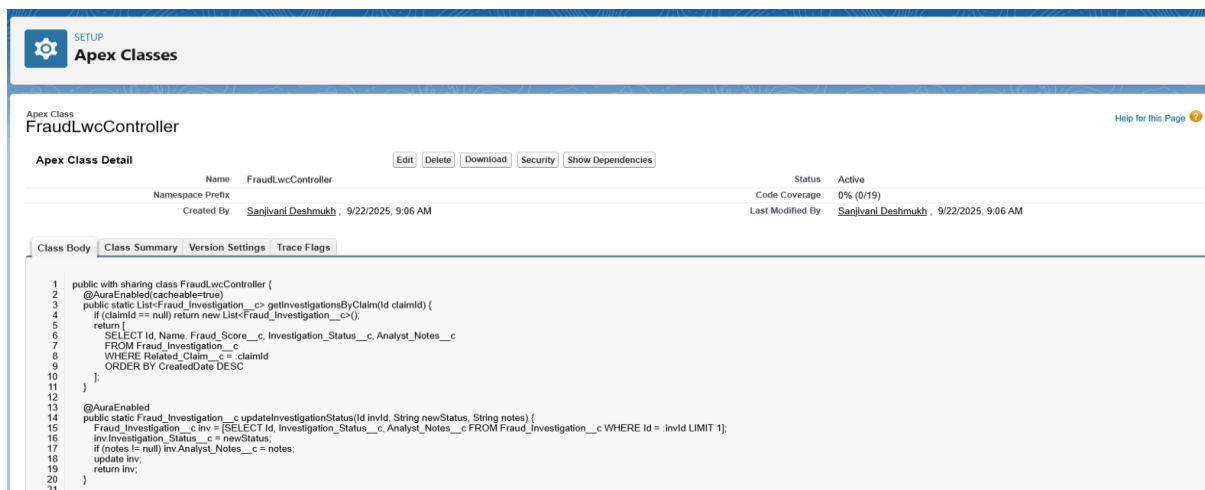
```
JS highRiskClaimsTable.js
force-app > main > default > lwc > highRiskClaimsTable > JS highRiskClaimsTable.js > ...
1  import { LightningElement, track } from 'lwc';
2  import getHighRiskClaims from '@salesforce/apex/FraudLwcController.getHighRiskClaims';
3  import { NavigationMixin } from 'lightning/navigation';
4
5  const actions = [{ label: 'Open', name: 'open' }];
6  const cols = [
7    { label: 'Claim #', fieldName: 'Claim_Number__c' },
8    { label: 'Amount', fieldName: 'Claim_Amount__c', type: 'currency' },
9    { label: 'Risk', fieldName: 'Risk_Score__c', type: 'number' },
10   { type: 'action', typeAttributes: { rowActions: actions } }
11 ];
12
13 export default class HighRiskClaimsTable extends NavigationMixin(LightningElement) {
14   @track claims = [];
15   columns = cols;
16
17   connectedCallback() {
18     getHighRiskClaims({ minRisk: 70 })
19       .then(result => { this.claims = result; })
20       .catch(err => { console.error(err); });
21   }
22
23   handleRowAction(event) {
24     const actionName = event.detail.action.name;
25     const row = event.detail.row;
26     if (actionName === 'open') {
27       this[NavigationMixin.Navigate]({
28         type: 'standard__recordPage',
29         attributes: { recordId: row.Id, objectApiName: 'Insurance_Claim__c', actionName: 'view'
30       });
31     }
32   }
33 }
```



```
highRiskClaimsTable.js-meta.xml X
force-app > main > default > lwc > highRiskClaimsTable > highRiskClaimsTable.js-meta.xml > ...
1 <?xml version="1.0" encoding="UTF-8"?>
2 <LightningComponentBundle xmlns="http://soap.sforce.com/2006/04/metadata">
3   <apiVersion>64.0</apiVersion>
4   <isExposed>true</isExposed>
5   <targets>
6     <!-- Can be added on App Pages -->
7     <target>lightning__AppPage</target>
8
9     <!-- Can be added on Home Page -->
10    <target>lightning__HomePage</target>
11
12    <!-- Can be added on Record Pages -->
13    <target>lightning__RecordPage</target>
14
15    <!-- Can be added in Utility Bar -->
16    <target>lightning__UtilityBar</target>
17  </targets>
18 </LightningComponentBundle>
19
```

5. Apex with LWC Integration

- Created Apex Controllers (**FraudLwcController**) to fetch claim and investigation data.
- Granted Apex Class Access to Analyst and Manager profiles.
- **Used:**
 - LWC cannot directly access Salesforce objects beyond Lightning Data Service; Apex ensures complex queries and business logic can be handled.
- **Steps:**
 1. Setup → Apex Classes → Created controllers.
 2. Setup → Profiles → Added Apex Class Access.



6. Events in LWC

- Used Custom Events (**investigationupdated**) in **fraudInvestigationPanel** to notify parent components.
- **Used:**
 - Enables real-time refresh of data without reloading the page.

7. Wire Adapters & Imperative Calls

- **Wire Adapters:**
 - Used **@wire** with Apex methods to fetch claim and investigation data.
 - Why: Automatic refresh when underlying data changes.
- **Imperative Calls:**
 - Used for on-demand operations like creating investigations.
 - Why: Offers flexibility when updates should only happen after user action.

8. Navigation Service

- Used **NavigationMixin.Navigate** to navigate from **highRiskClaimsTable** to claim record page.
- **Used:**
 - Improves usability, allowing users to jump directly to claim details.

9. Permissions & Profiles

- Granted Apex Class Access and LWC access to Analyst and Manager profiles.
 - Mapped Lightning Record Pages and Home Page to Fraud Detection App.
 - **Used:**
 - To ensure only authorized users (Analyst/Manager) can access fraud-related functionality.
-